

```
backend/
├── src/
│   ├── core/           # config, logging, exceptions
│   ├── db/             # session, migrations (Alembic)
│   ├── models/         # SQLAlchemy models (entities)
│   ├── schemas/        # Pydantic request/response
│   ├── services/       # business logic
│   │   ├── auth_service.py
│   │   ├── user_service.py
│   │   ├── track_service.py
│   │   ├── question_service.py
│   │   ├── submission_service.py
│   │   ├── adaptive_engine.py # ← core intelligence
│   │   ├── ai_service.py     # ← OpenAI wrapper
│   │   └── execution_service.py # ← Docker sandbox
└── client
    ├── api/
    │   └── v1/
    │       ├── routes/      # controllers
    │       ├── utils/
    │       └── main.py
    ├── services/           # Future microservices
    └── execution-service/  # Separate FastAPI app (Docker)
├── tests/
├── docker-compose.yml
├── alembic/               # migrations
└── README.md
```

For the above monolithic approach, produce one script for each folder within src/ (db, models, api/v1, etc.) and populate all the services scripts. Keep it short, minimal, and simple since I only want to understand how everything fits together and use it as a reference or guide for future projects

Below is a **minimal, reference-style monolithic FastAPI backend** based on your structure. Everything is intentionally simple so you can see how pieces connect.

src/core/config.py

</> Python



```
class Settings(BaseSettings):
    DATABASE_URL: str = "sqlite:///./app.db"
    OPENAI_API_KEY: str = "your-key"
    DEBUG: bool = True

settings = Settings()
```

src/core/exceptions.py

</> Python



```
class NotFoundException(HTTPException):
    def __init__(self, detail="Resource not found"):
        super().__init__(status_code=404, detail=detail)
```

src/core/logging.py

</> Python



```
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("app")
```

src/db/session.py

</> Python



```
from sqlalchemy.orm import sessionmaker, declarative_base
from src.core.config import settings
```

```
engine = create_engine(settings.DATABASE_URL, echo=settings.DEBUG)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

src/models/user.py

</> Python



```
from src.db.session import Base

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    email = Column(String, unique=True, index=True)
    password = Column(String)
```

src/models/question.py

</> Python



```
from src.db.session import Base

class Question(Base):
    __tablename__ = "questions"
    id = Column(Integer, primary_key=True)
    title = Column(String)
    difficulty = Column(String)
```

src/models/submission.py

</> Python



```
from src.db.session import Base

class Submission(Base):
    __tablename__ = "submissions"
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey("users.id"))
    question_id = Column(Integer, ForeignKey("questions.id"))
    status = Column(String)
```

src/schemas/user.py

</> Python



```
class UserCreate(BaseModel):
    email: str
    password: str

class UserOut(BaseModel):
    id: int
    email: str

class Config:
    orm_mode = True
```

src/schemas/question.py

</> Python



```
class QuestionOut(BaseModel):
    id: int
    title: str
    difficulty: str

class Config:
    orm_mode = True
```

src/schemas/submission.py

</> Python



```
class SubmissionCreate(BaseModel):
    question_id: int
    code: str

class SubmissionOut(BaseModel):
    id: int
    status: str

class Config:
    orm_mode = True
```

src/services/auth_service.py

</> Python



```
from src.models.user import User

def register(db: Session, email: str, password: str):
    user = User(email=email, password=password)
    db.add(user)
    db.commit()
    db.refresh(user)
    return user
```

src/services/user_service.py

</> Python



```
from src.models.user import User

def get_user(db: Session, user_id: int):
    return db.query(User).filter(User.id == user_id).first()
```

src/services/question_service.py

</> Python



```
from src.models.question import Question

def list_questions(db: Session):
    return db.query(Question).all()
```



src/services/submission_service.py

</> Python



```
from src.models.submission import Submission
from src.services.execution_service import execute_code

def submit(db: Session, user_id: int, question_id: int, code: str):
    result = execute_code(code)
    submission = Submission(
        user_id=user_id,
        question_id=question_id,
        status=result
    )
    db.add(submission)
    db.commit()
    db.refresh(submission)
    return submission
```



src/services/adaptive_engine.py

</> Python



```
if last_result == "passed":
    return "medium"
return "easy"
```



src/services/ai_service.py

</> Python



```
def generate_hint(question_title: str):  
    return f"Think about edge cases in {question_title}"
```

src/services/execution_service.py

</> Python



```
# placeholder for Docker sandbox call  
if "return" in code:  
    return "passed"  
return "failed"
```

src/services/track_service.py

</> Python



```
if user_level == "beginner":  
    return "basics"  
return "advanced"
```

src/api/v1/routes/user_routes.py

</> Python



```
from sqlalchemy.orm import Session  
from src.db.session import get_db  
from src.schemas.user import UserCreate, UserOut  
from src.services.auth_service import register  
  
router = APIRouter()  
  
@router.post("/", response_model=UserOut)  
def create_user(user: UserCreate, db: Session = Depends(get_db)):      
    return register(db, user.email, user.password)
```

src/api/v1/routes/question_routes.py

</> Python



```
from sqlalchemy.orm import Session
from src.db.session import get_db
from src.services.question_service import list_questions

router = APIRouter()

@router.get("/")
def get_questions(db: Session = Depends(get_db)):
    return list_questions(db)
```

src/api/v1/routes/submission_routes.py

</> Python



```
from sqlalchemy.orm import Session
from src.db.session import get_db
from src.schemas.submission import SubmissionCreate
from src.services.submission_service import submit

router = APIRouter()

@router.post("/")
def create_submission(data: SubmissionCreate, db: Session = Depends(get_db)):
    return submit(db, user_id=1, question_id=data.question_id, code=data.code)
```

src/api/v1/api.py

</> Python



```
from src.api.v1.routes import user_routes, question_routes, submission_routes

api_router = APIRouter()
api_router.include_router(user_routes.router, prefix="/users", tags=["Users"])
api_router.include_router(question_routes.router, prefix="/questions", tags=["Que
api_router.include_router(submission_routes.router, prefix="/submissions", tags=[
```


</> Python

”



```
from src.db.session import Base, engine
from src.api.v1.api import api_router

Base.metadata.create_all(bind=engine)

app = FastAPI(title="Monolith Example")

app.include_router(api_router, prefix="/api/v1")
```

How Everything Connects (High-Level)

</> Code



```
→ Services (Business Logic)
→ Models (DB layer)
→ Database

Submission Flow:
Route → submission_service
      → execution_service (Docker)
      → adaptive_engine
      → DB save
```

What This Gives You

- Clean separation of concerns
- Easy to split into microservices later
- Clear service layer pattern
- Minimal boilerplate
- Good reference architecture

If you'd like, I can now:

- Convert this into a **production-grade version**
- Show how to split execution-service into a microservice

- Add JWT authentication properly
- Add dependency injection pattern
- Add test examples
- Or generate a visual architecture diagram



When building an application of this architecture, where do you first start?

Great question — this is what separates clean architecture from chaos.

With **this exact structure**, here is the correct order to build it in real life:

1. Start With the Domain (Models + Core Logic)

Always start with what your system *is*, not with routes or UI.

Build first:

- `models/`
- Core service logic (`services/` without external integrations)
- Basic schemas

Ask yourself:

- What entities exist? (User, Question, Submission)
- What are their relationships?
- What are the core rules? (grading, adaptive difficulty, tracks)

Example:

- Define `User`
- Define `Question`
- Define `Submission`
- Write `adaptive_engine.py`

At this stage:

No FastAPI.

No routes.

No Docker.
No OpenAI.
Just pure Python logic.



👉 This keeps your brain focused on business rules.

2. Set Up Database Layer

Now wire persistence.

Build:

- `db/session.py`
- Connect SQLAlchemy
- Create tables
- Make simple CRUD in services

Test from Python shell before adding API.

Example:

 Python



```
create_user(...)  
list_questions(...)
```

Make sure DB works before exposing it.

3. Add Services Layer Properly

Now expand:

- `auth_service`
- `user_service`
- `question_service`
- `submission_service`

Keep them:

- Pure functions
- No FastAPI imports

- No request objects

Your services should be reusable outside HTTP.



4. Add API Layer (Routes)

Now add:

- `api/v1/routes/`
- `main.py`

Routes should:

- Validate input
- Call services
- Return responses

They should contain **zero business logic**.

Think of routes as “translation layer”:

HTTP ↔ Python



5. Add External Integrations

Now add:

- `execution_service.py` (Docker client)
- `ai_service.py` (OpenAI wrapper)
- Logging
- Config management

These are infrastructure concerns.

They should plug into services — not control them.



6. Add Tests

Test in this order:

1. Services (unit tests)
2. DB integration tests
3. API tests

Never start by testing routes.



Why This Order Works

You are building:

</> Code



```
↓
Persistence
↓
Application Services
↓
Transport Layer (API)
↓
External Integrations
```

This prevents:

- Tight coupling
- Rewrite hell
- Microservice migration pain
- Hard-to-test systems



What NOT To Do

Don't start with:

- FastAPI routes
- Docker
- JWT
- OpenAI
- Frontend

That creates:

"Controller-driven architecture" (bad).



Mental Model

Think like this:

If I delete FastAPI today, does my business logic still work?

If yes → good architecture.



In Your Specific Case (Adaptive Coding Platform)

Your real starting point should be:

1. Question model
2. Submission model
3. `adaptive_engine.py`
4. `execution_service.py` (mocked)
5. Submission grading flow

Because that's your core product.

Users and auth are secondary.

Final Answer

Start here:

 Code



- ```
2. Core business logic
3. Database wiring
4. Services
5. API layer
6. Infrastructure & integrations
7. Tests
```

---

If you'd like, I can also explain:

- How this evolves into microservices
- How senior engineers approach greenfield builds
- Or how to validate architecture before writing code



